

详细技术方案：专家模型稳定产出 IR → 大模型生成/修复 React (Figma Design-to-Code + 视觉校验闭环)

日期：2025-1-24

版本：v1.0 (技术方案详细版)

目标：在不依赖设计 Auto Layout 的情况下，实现稳定的布局/资产/语义判断；在此基础上用大模型生成组件库级 React 代码，并通过视觉 diff 驱动自动修复，形成可规模化的闭环系统。

1. 设计原则与总体思路

1.1 核心原则

- 稳定性优先：让“可重复、可回归”的模块在前（专家模型/规则/校验），让“强推理、长尾处理”在后（大模型）。
- 结构化输入输出：减少把原始 Figma JSON 直接喂给大模型，避免噪音与 token 膨胀。
- 可验证/可回退：每一步产物都必须能被 schema 校验、被静态分析/构建验证；失败可回退到更保守策略（导图/降级 View）。
- 闭环沉淀数据资产：将 schema、IR、代码、渲染图、diff、patch、用户反馈归档，支撑后续模型迭代与专家化降本。

1.2 分层架构（关键）

- 规则/几何算法层 (Deterministic)：从 Figma schema 提取 bbox、层级、重叠、复杂度等确定性特征，产出初始建议。
- 专家模型 (Stable Expert)：把“特征+局部上下文”归一成稳定的 IR (Intermediate Representation)，做三类决策：
 - 图片合成/导出策略 (codeable vs export_png/svg)
 - 背景图判别与实现策略 (css background vs img)
 - 无 Auto Layout 的布局推断 (row/column/grid/absolute + gap/align)
- 大模型 (Reasoning LLM)：消费 IR + 组件库约束，生成高质量 React 代码；消费 diff 报告生成 patch 修复。
- 质量守门员 (CI-like Gate)：typecheck/lint/build/render/diff，为“效果”和“稳定”提供硬约束。

2. 目标能力与范围

2.1 MVP 能力（6-10 周）

- Figma 选中 Frame → 输出稳定 IR (布局/语义/资产策略)
- IR → 生成可运行 React 代码 (组件库级：推荐 Ant Design)
- Playwright 渲染截图 → 与 Figma baseline 做 pixel diff
- mismatch 超阈值 → 自动 patch 修复 (最多 2 轮)
- 全量归档：schema/IR/code/baseline/render/diff/patch/feedback

2.2 暂不支持 (MVP)

- 复杂交互 (动画、多页面路由编排)
- 像素级 0 diff (目标是可控误差+可修复)
- 全套 design system 治理

3. 系统架构与组件

3.1 组件列表

- Figma Plugin (TS)

- 读取选中 Frame node tree
- 导出 baseline PNG（用于视觉对比）
- 上传 schema + baseline + options 到后端
- 展示结果：代码预览/下载、diff、修复按钮

2. Backend Orchestrator (Node.js / Python)

- Job 管理、队列与状态（MVP 可先无队列）
- 特征提取（bbox、重叠、复杂度）
- 专家模型推理生成 IR
- 大模型 codegen & repair
- build/render/diff
- 归档与指标统计

3. Runner (Playwright + pixelmatch/OpenCV)

- 固定 viewport/DPR 渲染
- 生成 render.png
- 生成 diff heatmap + mismatch%

4. Artifact Store

- 本地磁盘 + SQLite（MVP）
- 结构化归档以便训练与回归

3.2 数据流

Figma Frame

→ Plugin: {figmaSchema, baselinePng}
 → Backend: feature extraction
 → Expert Model: IR
 → LLM: React code
 → Build/Render: renderPng
 → Diff: mismatch + heatmap
 → LLM: patch (≤ 2 rounds)
 → Archive + metrics

4. 输入与中间表示：IR (Intermediate Representation)

4.1 为什么要 IR（而不是只用 DSL）

IR 的目标是“稳定可控”：输出空间小、可校验、可离散化，让专家模型更容易收敛。

4.2 IR 内容（建议三块）

1. **LayoutIR**：容器树 + 布局策略（flex/grid/absolute/overlay）
2. **SemanticIR**：节点语义标签 + 置信度 + 可选 props
3. **AssetIR**：图片/矢量/背景导出与实现策略

4.3 IR JSON 示例

```
{
  "ir_version": "0.2",
  "frame": { "name": "Login", "w": 1440, "h": 900 },
  "layout_tree": {
    "id": "root",
    "layout_strategy": "flex-column",
    "align": "center",
    "justify": "center",
    "gap": 16,
    "children": [
      {
        "id": "card1",
        "semantic_type": "Card",
        "confidence": 0.92,
        "layout_strategy": "flex-column",
        "gap": 12,
        "padding": [24,24,24,24],
        "children": [
          { "id": "title", "semantic_type": "Heading", "confidence": 0.88, "text": "Sign in" },
          { "id": "email", "semantic_type": "TextField", "confidence": 0.81, "label": "Email" },
          { "id": "pwd", "semantic_type": "TextField", "confidence": 0.79, "label": "Password" },
          { "id": "btn", "semantic_type": "PrimaryButton", "confidence": 0.86, "text": "Login" }
        ]
      }
    ]
  },
  "assets": [
    { "id": "bg", "role": "background", "decision": "css-background", "confidence": 0.9 },
    { "id": "heroDecor", "role": "decorative", "decision": "export_png", "confidence": 0.84 }
  ],
  "warnings": [
    { "code": "LOW_CONFIDENCE", "nodeId": "pwd", "confidence": 0.79 }
  ]
}
```

4.4 输出空间（枚举约束）

为稳定性，限定枚举集合（可版本化扩展）：

- layout_strategy ∈ { flex-row, flex-column, grid, absolute, overlay, unknown }
- semantic_type ∈ { View, Text, Heading, Paragraph, Button, PrimaryButton, SecondaryButton, TextField, Checkbox, Radio, Select, Card }
- asset.decision ∈ { codeable, export_png, export_svg, css-background, img-tag }

并对数值字段做离散/Token 化：

- gap/padding：按 2/4/8 的倍数或 spacing tokens (xs/sm/md/lg)

5. 规则/几何算法层（专家模型的“先验”）

专家模型不从零猜，而是在确定性算法基础上做归一与纠错，显著提升稳定性。

5.1 特征提取（Feature Extraction）

从 Figma schema 提取：

- bbox: x,y,w,h
- z-order: 层级顺序
- 类型: TEXT/RECTANGLE/VECTOR/BOOLEAN/IMAGE/FRAME/GROUP
- effects: shadow/blur

- paint: solid/gradient/image
- mask、blendMode、opacity
- 节点名（命名启发）

计算派生特征：

- 覆盖率：area(node)/area(parent)
- 重叠度：IoU 与重叠计数
- 复杂度分数：mask/boolean/gradient/blur/path count 等加权
- 对齐指标：left/right/center 的偏差统计
- 间距分布：相邻元素 gap 的均值与方差

6. 专家模型：任务拆分与实现方案

6.1 专家模型建议做“多任务小输出”

不要训一个“Figma→React 大一统模型”，而训（或一个共享骨干的多头网络）：

1. Asset Decision Head

- 输出：codeable/export_png/export_svg/css-background/img-tag
- 输入：结构复杂度特征 + 层级 + 局部截图 embedding（可选）

2. Background Role Head

- 输出：background/decorative/content
- 输入：覆盖率、z-order、父子关系、命名、重叠统计、截图特征

3. Layout Strategy Head

- 输出：flex-row/flex-column/grid/absolute/overlay
- 回归/离散输出：gap、padding、align、justify、columns (grid)
- 输入：bbox 聚类结果、对齐/间距统计、层级结构

4. Semantic Tag Head（可选 MVP）

- 输出：Button/TextField/Card/Heading/...
- 输入：节点类型、文本内容模式、尺寸、相邻关系 (label+input)、命名启发

6.2 无 Auto Layout 布局推断（核心算法）

1. 行/列聚类：
 - y 聚类→行；x 聚类→列（DBSCAN 或阈值聚类）
2. gap 推断：
 - 在排序后的元素间计算 gap，取众数/中位数并离散到 tokens
3. 对齐推断：
 - 统计 left edges / center / right edges 的方差，最小者为对齐方式
4. grid 推断：
 - 同时存在稳定行簇与列簇 + cell 尺寸相近→grid
5. 容器推断：
 - bbox containment 建树，递归对局部容器推断布局

输出：layout_tree 的 layout_strategy + 参数

6.3 图片合成判断（codeable vs export）

用复杂度评分 + 规则：

- 强烈建议 export_png：
 - mask + blur / 多层 blendMode / boolean ops / 多层渐变叠加 / 重重叠

- 倾向 codeable:
 - solid + 简单圆角 + 单层阴影 + 简单 icon (svg)

专家模型负责融合、输出稳定决策与置信度。

6.4 背景图判断

规则优先：

- 覆盖率高、处于底层、上面叠加多元素 → background
专家模型处理边界：区块背景、卡片背景、装饰图等。

7. 大模型：IR → React Code（组件库级）+ Repair

7.1 目标技术栈（建议）

- React + TypeScript + Vite
- Ant Design (MVP)
- CSS Modules (MVP) 或 Tailwind (后续)

7.2 输入规范（节省 token）

大模型输入仅包含：

- IR (layout_tree + assets + warnings)
- design tokens (colors/spacing/typography)
- 组件白名单与映射表 (semantic_type → AntD 组件)
- 输出格式要求 (多文件 JSON)

不输入完整 Figma schema (除非 debug/失败回退)。

7.3 输出规范

```
{
  "files": [
    { "path": "src/pages/Login.tsx", "content": "...", },
    { "path": "src/pages/login.module.css", "content": "...", }
  ]
}
```

7.4 Repair (diff 驱动 patch)

输入：

- IR
- 当前代码文件
- mismatch% 与 diff 关键区域摘要 (可选：最大连通区域 bbox)

输出：

- patch (推荐 unified diff) 或更新后的 files

限制：

- 最多 2 轮
- 修复范围先聚焦布局/样式 (padding/gap/align/typography)

8. 质量守门员与回退策略

8.1 守门员（硬约束）

- JSON schema 校验：IR 与 LLM 输出
- TypeScript typecheck
- ESLint/Prettier
- build 成功
- Playwright 渲染成功
- diff 指标输出成功

8.2 回退策略（确保可交付）

- 若大模型生成失败：
 - 回退到更保守模板（View + 样式最小化）
- 若 mismatch 过高且无法修复：
 - 触发 Asset Decision：将局部区域导出为 PNG 并嵌入（保证“像”）
- 若语义置信度低：
 - 降级为 View，保持布局正确

9. API 设计（最小可用）

9.1 创建任务

POST /jobs

```
{
  "figmaSchema": { /* plugin extracted */ },
  "baselinePngBase64": "...",
  "options": {
    "uiKit": "antd",
    "style": "css-modules",
    "repairRounds": 2
  }
}
```

9.2 查询任务

GET /jobs/:id

返回状态、指标、产物链接、IR 摘要。

9.3 下载产物

GET /jobs/:id/artifacts/:name

- baseline.png / render.png / diff.png
- code.zip / patch.diff
- ir.json / metrics.json

10. 数据归档与训练闭环

10.1 归档内容（每 job）

- figma_schema.json
- baseline.png

- ir.json
- code_round0/*
- render_round0.png
- diff_round0.png
- patch_round1.diff （如有）
- code_round1/* （如有）
- metrics.json
- feedback.json （可选：用户接受/修改点）

10.2 专家模型训练数据构建（推荐策略）

- 弱标注：用规则先生成标签（background/export/layout）
- 人工抽检：对低置信度/高影响样本标注
- 训练目标：分类为主、少量参数回归为辅（离散化）

11. 成本与效果（关键决策说明）

11.1 为什么“专家模型先输出 IR，再喂大模型”更好

- **效果**：大模型在“干净 IR + 强约束”下更容易生成符合规范的代码
- **稳定**：IR 输出空间小，易做回归测试；减少大模型随机性影响
- **成本**：IR 比原始 schema 短很多，显著减少 token；高频决策可在专家模型本地完成
- **私有化**：专家模型可先私有化落地，敏感数据不出域；大模型可替换为内网 LLM

11.2 适用阶段

- MVP：专家模型可先由规则+轻量模型替代；大模型承担主要生成
- 规模化：逐步把高频决策与骨架生成迁移到专家模型，降低 API 成本

12. 实施计划（6–10 周）

- W1–W2：Plugin 抽取 + 特征提取 + IR schema 定义 + job/归档
- W3–W4：规则/几何布局推断（无 Auto Layout）+ IR 生成（先无模型或轻量模型）
- W5：接入大模型 codegen（IR→React），建立 build/render 流水线
- W6：pixel diff 报告输出 + 指标体系
- W7–W8：repair loop（diff→patch）+ 回退策略（导出 png）
- W9–W10：引入专家模型（可选）/增强分类与布局推断，扩展组件覆盖

13. 验收指标（MVP）

- 可编译率 $\geq 90\%$
- 结构合理率（人工抽检） $\geq 70\%$
- mismatch%：
 - 50% 页面 $< 3\%$
 - 80% 页面 $< 8\%$
- 修复有效率（触发修复样本） $\geq 60\%$
- 成本指标：
 - 平均 tokens/job、P50/P95 延迟、平均 repair 轮次

14. 风险与对策

1. 无 Auto Layout 推断错误
- 对策：几何算法优先 + 置信度阈值降级 + 允许局部 absolute
2. diff 误报（字体/抗锯齿）
- 对策：固定 viewport/DPR/字体；后续 mask/容差与分区权重
3. 大模型偶发不合规输出
- 对策：白名单、schema 校验、typecheck/lint、失败回退模板
4. 成本随规模上升
- 对策：IR 缩短上下文 + 缓存 + 专家模型承担高频决策与骨架生成

15. 待确认（1 周内必须定）

1. 组件库：Ant Design / MUI / 自研？
2. 样式策略：CSS Modules / Tailwind？
3. 目标页面：B 端后台表单为主 or C 端营销页？
4. 输出粒度：单页单文件 or 自动拆分子组件？
5. design tokens 是否已有？spacing/color/typography 的来源与命名规范？
6. 部署：云端内部工具 or 必须私有化？

16. 附录：推荐的 IR 枚举（MVP 精简版）

- layout_strategy: flex-row | flex-column | grid | absolute | overlay | unknown
- semantic_type (先 20 个): View, Text, Heading, Paragraph, Button, PrimaryButton, SecondaryButton, TextField, Checkbox, Radio, Select, Card, List, ListItem, ...
- asset_decision: codeable | export_png | export_svg | css-background | img-tag

结论：采用“规则/几何 + 专家模型稳定 IR → 大模型生成/修复”的分层方案，可在 MVP 阶段快速落地并显著提升稳定性与可控性，同时为规模化降本与私有化部署预留清晰路径。